

Contents

1	Introduction	2
2	Methodology	3
2.1	Lexical Analysis	3
2.2	Syntax Analysis and Context-Free Grammar	4
2.3	Symbol Table Management	7
2.4	Intermediate Code Generation	8
2.5	Assembly Code Generation	9
3	Results	10
3.1	Input Program	10
3.2	Symbol Table Output	11
3.3	Intermediate Code	11
3.4	Generated Assembly (Key Fragments)	11
3.5	Program Output	13
4	Discussion	13
4.1	Lexical and Syntactic Analysis	13
4.2	Semantic Analysis and Symbol Table	14
4.3	Intermediate and Assembly Code Generation	14
4.4	Design and Extensibility	14
5	Conclusion	14
5.1	Achievements	15
5.2	Validation	15
5.3	Future Directions	15

Design of a Simple Compiler for a Domain-Specific Language

Roll: 2003055

Name: Eazdan Mostafa Rafin

1 Introduction

This section provides a theoretical overview of compiler design and the objectives of the current project. The report discusses lexical analysis, syntax analysis with context-free grammar, symbol table management, intermediate code generation, assembly code generation, and presents detailed results and validation of the implementation.

Compilation Phases: A compiler translates a high-level source program into a target machine program through a sequence of well-defined phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and final code generation.

Language and Implementation: In this work, a simplified compiler front-end and back-end were implemented for a small, domain-specific language that uses Bangla-inspired keywords and supports a single `main` function, integer variable declarations, assignment statements, integer addition, and output statements.

Design Methodology: The implementation follows the standard design of a one-pass, syntax-directed translator where the lexical analyser produces tokens, the syntax analyser validates the context-free grammar (CFG), and semantic actions build an intermediate representation while interacting with a symbol table.

Intermediate and Target Representation: The intermediate code is a simple stack-based, three-address-style code over a small instruction set (`START`, `HALT`, `LD_INT`, `LD_VAR`, `STORE`, `ADD`, `PRINT_INT_VALUE`, etc.), which is systematically translated into MASM-style x86 assembly instructions.

Project Objectives: The goal of the project is to design and implement lexical analysis, syntax analysis, intermediate code generation, and final code generation for a given example program written in this domain-specific language.

```
1 main function ()
2
3 suru
4
5     dhorì x ar type integer
6     dhorì y ar type integer
```

```
7      x soman 2
8      y soman 3 jog x
9      print kori y
10
11 sesh
```

Listing 1: Source program in the domain-specific language

2 Methodology

This section describes the implementation of each compiler phase: lexical analysis, syntax analysis, intermediate code generation, symbol table management, and final assembly generation.

2.1 Lexical Analysis

The lexical analyser was implemented using Flex in the file `lexer.1`. The scanner defines regular expressions for identifiers, integer constants, and reserved words. Relevant excerpts are shown in Listing 2.

```
1 alpha      [a-zA-Z]
2 digit      [0-9]
3 alnum      {alpha}{digit}
4
5 ID          {alpha}{alnum}*
6 ICONST     [0-9]{digit}*
7 %%
8 "main"[ \t]+"function"[ \t]*("[ \t]*")"    { return MAIN_FUNC; }
9 "suru"                                           { return SURU; }
10 "sesh"                                          { return SESH; }
11 "dhori"                                        { return DHORI; }
12 "ar"[ \t]+"type"                             { return AR_TYPE; }
13 "integer"                                     { yylval.int_val=
      INT_TYPE; return INTEGER; }
14 "soman"                                       { return SOMAN; }
15 "jog"                                        { return JOG; }
16 "print"[ \t]+"kori"                         { return PRINT_KORI; }
17
18 {ID}      {strcpy(yylval.str_val, yytext); return ID;}
19 {ICONST}  {yylval.int_val=atoi(yytext); return ICONST;}
20
21 "//".* { }
```

```
22 "\n"          { lineno += 1; }
23 [ \t\r\f]+
24
25 .             { yyerror("Unrecognized character"); }
```

Listing 2: Excerpt from the lexical analyser (lexer.l)

The scanner associates lexical values with tokens using the `yylval` union (string for identifiers, integer value for constants). Lexical errors (unrecognized characters) are reported with `yyerror`.

A sample token mapping table for the given input program is shown in Table 1.

Table 1: Token mapping for the sample program

Lexeme	Token	Attribute / Value
main function ()	MAIN_FUNC	—
suru	SURU	—
dhori	DHORI	—
x	ID	"x"
ar type	AR_TYPE	—
integer	INTEGER	INT_TYPE (1)
dhori	DHORI	—
y	ID	"y"
ar type	AR_TYPE	—
integer	INTEGER	INT_TYPE (1)
x	ID	"x"
soman	SOMAN	—
2	ICONST	2
y	ID	"y"
soman	SOMAN	—
3	ICONST	3
jog	JOG	—
x	ID	"x"
print kori	PRINT_KORI	—
y	ID	"y"
sesh	SESH	—

2.2 Syntax Analysis and Context-Free Grammar

The syntax analyser was implemented using Bison/Yacc in `parser.y`. The grammar formalises the structure of programs in the small language and embeds semantic actions

for symbol-table insertion and code generation. The relevant part of the grammar is shown in Listing 3.

```
1 %start program
2
3 %token MAIN_FUNC SURU SESH DHORI AR_TYPE SOMAN PRINT_KORI JOG
4 %token<str_val> ID
5 %token<int_val> ICONST
6 %token<int_val> INTEGER
7
8 %left JOG
9 %type<int_val> exp
10
11 %%
12
13 program: {gen_code(START, -1);} MAIN_FUNC SURU code SESH {gen_code
14         (HALT, -1);}
15         ;
16
17 code: statements;
18
19 statements: statements statement | ;
20
21 statement: declaration
22           | assignment
23           | print_statement
24           ;
25
26 declaration: DHORI ID AR_TYPE INTEGER
27             {
28               insert($2, $4);
29             }
30             ;
31
32 assignment: ID SOMAN exp
33            {
34              int address = idcheck($1);
35
36              if(address != -1)
37              {
38                gen_code(STORE, address);
39              }
```

```
39         else
40             yyerror();
41     }
42     ;
43
44 print_statement: PRINT_KORI ID
45     {
46         int address = idcheck($2);
47
48         if(address != -1)
49         {
50             gen_code(PRINT_INT_VALUE, address);
51         }
52         else
53             yyerror();
54     }
55     ;
56
57 exp: ICONST
58     {
59         gen_code(LD_INT, $1);
60     }
61     | ID
62     {
63         int address = idcheck($1);
64
65         if(address != -1)
66         {
67             gen_code(LD_VAR, address);
68         }
69         else
70             yyerror();
71     }
72     | exp JOG exp { gen_code(ADD, -1); }
73     ;
```

Listing 3: Excerpt from the syntax analyser (parser.y)

From this, an equivalent context-free grammar (CFG) without semantic actions can

be written as:

$$\begin{aligned} \text{program} &\rightarrow \text{MAIN_FUNC SURU code SESH} \\ \text{code} &\rightarrow \text{statements} \\ \text{statements} &\rightarrow \text{statements statement} \mid \epsilon \\ \text{statement} &\rightarrow \text{declaration} \mid \text{assignment} \mid \text{print_statement} \\ \text{declaration} &\rightarrow \text{DHORI ID AR_TYPE INTEGER} \\ \text{assignment} &\rightarrow \text{ID SOMAN exp} \\ \text{print_statement} &\rightarrow \text{PRINT_KORI ID} \\ \text{exp} &\rightarrow \text{ICONST} \mid \text{ID} \mid \text{exp JOG exp} \end{aligned}$$

2.3 Symbol Table Management

The symbol table is implemented in `syntab.c` as a singly linked list of entries, each recording name, type, and address. New identifiers are inserted during declarations, and lookups are performed when identifiers appear in expressions or assignments. An excerpt is shown in Listing 4.

```
1 list_t *head = NULL;
2 int address = 0;
3 extern int lineno;
4
5 void insert(char* name, int type)
6 {
7     if(search(name)==NULL)
8     {
9         list_t* temp = (list_t*)malloc(sizeof(list_t));
10
11         strcpy(temp->st_name, name);
12         temp->st_type = type;
13         temp->address = address;
14         address++;
15
16         printf("In line no %d, Inserting %s with type %s in symbol
17         table.\n",
18                 lineno, name, typename[type]);
19
20         temp->next = head;
21         head = temp;
22     }
```

```
22     else
23     {
24         printf("In line no %d, Same variable %s is declared more
25         than once.\n",
26                 lineno, name);
27     }
```

Listing 4: Excerpt from symbol table implementation (syntab.c)

2.4 Intermediate Code Generation

Intermediate code is generated in a syntax-directed manner from the parser actions using the routines in `codeGen.c`. The intermediate instruction representation is based on a small set of operations such as `START`, `HALT`, `LD_INT`, `LD_VAR`, `STORE`, `ADD`, and `PRINT_INT_VALUE`. An excerpt is shown in Listing 5.

```
1 void gen_code(enum code_ops op, int arg)
2 {
3     code[code_offset].op = op;
4     code[code_offset].arg = arg;
5
6     code_offset++;
7 }
```

Listing 5: Intermediate code generation routine (codeGen.c)

The parser inserts appropriate instructions for program start and end, for expressions, assignments and print statements. For example, for expressions:

```
1 exp: ICONST
2     {
3         gen_code(LD_INT, $1);
4     }
5     | ID
6     {
7         int address = idcheck($1);
8
9         if(address != -1)
10        {
11            gen_code(LD_VAR, address);
12        }
13        else
14            yyerror();
```

```
15     }
16     | exp JOG exp { gen_code(ADD, -1); }
17     ;
```

Listing 6: Code generation for expressions in parser.y

2.5 Assembly Code Generation

The final code generation phase in `codeGen.c` converts the intermediate code sequence into MASM-style x86 assembly. The driver function `print_assembly()` iterates over the intermediate instructions and emits assembly according to the opcode. A condensed excerpt is shown in Listing 7.

```
1 void print_assembly()
2 {
3     int i = 0;
4     int stack_variable_counter = 0;
5
6     for(i=0; i<code_offset; i++)
7     {
8         printf("\n;%s %d\n", op_name[code[i].op], code[i].arg);
9
10        if(code[i].op == LD_INT || code[i].op == LD_VAR)
11            stack_variable_counter++;
12
13        if(code[i].op == ADD)
14            stack_variable_counter--;
15
16        switch(code[i].op)
17        {
18            case START:
19
20                printf(".686\n");
21                printf(".model flat, c\n");
22                /* ... prologue omitted ... */
23                break;
24
25            case HALT:
26
27                printf("\tadd ebp, 100\n");
28                printf("\tmov esp, ebp\n");
29                printf("\tpop ebp\n");
30                printf("\tret\n");
31                printf("main endp\n");
```

```
30         printf("end\n");
31         break;
32
33     case LD_INT:
34         printf("\tmov eax, %d\n", code[i].arg)
35         ;
36         printf("\tmov dword ptr [ebx], eax\n")
37         ;
38         printf("\tadd ebx, 4\n");
39         break;
40         /* Cases for LD_VAR, ADD, STORE, PRINT_INT_VALUE
41         omitted for brevity */
42         default:
43             break;
44     }
45 }
```

Listing 7: Excerpt from assembly code generation (codeGen.c)

3 Results

3.1 Input Program

The input program in `input.txt` is reproduced in Listing 8.

```
1 main function ()
2
3 suru
4
5     dhoru x ar type integer
6
7     dhoru y ar type integer
8
9     x soman 2
10
11     y soman 3 jog x
12
13     print kori y
14
15 sesh
```

Listing 8: Input program (input.txt)

3.2 Symbol Table Output

During compilation, the symbol table insertion messages for the two declarations are:

In line no 4, Inserting x with type integer in symbol table.

In line no 6, Inserting y with type integer in symbol table.

Here x receives address 0 and y receives address 1.

3.3 Intermediate Code

After a successful parse, the parser prints the intermediate code by calling `print_code()`.

For the given program, the expected sequence is:

1	0: START	-1	
2	1: LD_INT	2	; x soman 2
3	2: STORE	0	
4	3: LD_INT	3	; y soman 3 jog x
5	4: LD_VAR	0	
6	5: ADD	-1	
7	6: STORE	1	
8	7: PRINT_INT_VALUE	1	; print kori y
9	8: HALT	-1	

Listing 9: Intermediate code for the sample program

Instructions 1–2 initialise x to 2, instructions 3–6 compute $y = 3 + x$, and instruction 7 prints the value of y. The `START` and `HALT` instructions define the program prologue and epilogue.

3.4 Generated Assembly (Key Fragments)

Key fragments of the generated assembly, produced by `print_assembly()`, are shown below.

```
1 .686
2 .model flat, c
3 include C:\masm32\include\msvcrt.inc
4 includelib C:\masm32\lib\msvcrt.lib
5
6 .stack 100h
```

```
7 printf PROTO arg1:Ptr Byte, printlist:VARARG
8 scanf PROTO arg2:Ptr Byte, inputlist:VARARG
9
10 .data
11 output_integer_msg_format byte "% %d", 0Ah, 0
12 input_integer_format byte "% %d", 0
13 number sdword ?
14
15 .code
16
17 main proc
18     push ebp
19     mov  ebp, esp
20     sub  ebp, 100
21     mov  ebx, ebp
22     add  ebx, 4
```

Listing 10: Program prologue (assembly)

The assignments and print statement are compiled into the following instructions:

```
1 ; x soman 2
2     mov  eax, 2
3     mov  dword ptr [ebx], eax
4     add  ebx, 4
5     mov  dword ptr [ebp-0], eax
6
7 ; y soman 3 jog x
8     mov  eax, 3
9     mov  dword ptr [ebx], eax
10    add  ebx, 4
11
12    mov  eax, [ebp-0]
13    mov  dword ptr [ebx], eax
14    add  ebx, 4
15
16    sub  ebx, 4
17    mov  eax, [ebx]
18    sub  ebx, 4
19    mov  edx, [ebx]
20    add  eax, edx
21    mov  dword ptr [ebx], eax
22    add  ebx, 4
```

```
23
24     mov dword ptr [ebp-4], eax      ; y
25
26 ; print kori y
27     mov eax, [ebp-4]
28     INVOKE printf, ADDR output_integer_msg_format, eax
```

Listing 11: Assignments and print statement (assembly)

The program concludes with the epilogue:

```
1     add    ebp, 100
2     mov    esp, ebp
3     pop    ebp
4     ret
5 main endp
6 end
```

Listing 12: Program epilogue (assembly)

3.5 Program Output

At runtime, the semantics of the program are as follows:

1. `x` is initialised to 2.
2. `y` is initialised to $3 + x = 3 + 2 = 5$.
3. The statement `print kori y` displays the value of `y`.

Thus, when the compiled assembly is executed, the observable output corresponds to the integer value 5 printed on the console (up to formatting characters in the format string).

4 Discussion

The following analysis examines the correctness and design of each compilation phase and discusses the overall architecture.

4.1 Lexical and Syntactic Analysis

Lexical Recognition: The lexical analyser correctly recognises identifiers, integer literals and language keywords using concise regular expressions, and the token mapping

table confirms that lexemes are consistently associated with the intended token classes and attributes.

Grammar Unambiguity: The context-free grammar is unambiguous for the restricted construct set (declarations, assignments, print statements and left-associative addition), and the use of left associativity for the `JOG` token ensures a standard arithmetic evaluation order.

4.2 Semantic Analysis and Symbol Table

Symbol Table Role: Symbol table management is central to semantic correctness: insertion prevents redeclaration, and identifier lookup ensures that all variables used in expressions and assignments are declared, allowing early detection of semantic errors.

4.3 Intermediate and Assembly Code Generation

Stack-Based Intermediate Code: Intermediate code generation is tightly integrated with parsing via syntax-directed translation; every occurrence of `ICONST`, `ID` and `JOG` triggers the emission of `LD_INT`, `LD_VAR` and `ADD` instructions, realising a simple stack-based evaluation strategy.

Systematic Code Correspondence: The assembly code generator establishes a systematic correspondence between intermediate instructions and target instructions; although no optimisation is performed, the generated assembly preserves the intended semantics and correctly computes and prints the value of `y`.

4.4 Design and Extensibility

Complete Pipeline: The implementation realises a complete but minimal compilation pipeline from a high-level, domain-specific language down to x86 assembly, demonstrating the interaction of lexical analysis, parsing, semantic checking, intermediate code generation, and final code emission.

Modular Architecture: The modular structure of the phases makes the design extensible: additional statement types, data types, or optimisation passes could be added by extending the grammar, symbol table, and intermediate instruction set without changing the overall architecture.

5 Conclusion

This section summarises the key outcomes and the scope for future extensions.

5.1 Achievements

- **Complete Implementation:** The project successfully implemented a small but complete compiler for a restricted imperative language, covering lexical analysis, syntax analysis, intermediate code generation, and final code generation.
- **Tool Integration:** The combination of Flex and Bison allowed the construction of a scanner and parser that together validate source programs against a clearly specified context-free grammar while driving syntax-directed translation.
- **Semantic Enforcement:** The linked-list-based symbol table provided essential semantic checks, such as enforcing declaration-before-use and preventing duplicate declarations, thereby improving the robustness of the compiled programs.
- **Abstraction Layers:** The stack-based intermediate code representation abstracted away low-level machine details while remaining close enough to be translated systematically into MASM-style x86 assembly.

5.2 Validation

Correctness Verification: For the given input program, the compiled code correctly computes $y = 3 + x$ and prints the value **5**, providing empirical validation of the correctness of the compilation pipeline.

5.3 Future Directions

Extensibility Potential: The implementation provides a concrete foundation on which more advanced compiler features can be built, including richer type systems, control-flow constructs, and optimisation passes operating over the intermediate representation.